

推荐算法在Top-N推荐任务上的性能

保罗Cremonesi

米兰理工大学

米兰, 意大利

Yehuda Koren

雅虎研究

以色列海法

Roberto Turrin

Neptunus

米兰, 意大利roberto.

turrin@polimi.it

摘要

在许多商业系统中, 会显示“最佳赌注”的推荐, 但不会显示预测的评级值。这通常被称为top-N推荐任务, 其中推荐系统的目标是找到几个应该对用户最有吸引力的特定项目。基于误差度量(如RMSE)的常用方法并不适合评估top-N推荐任务。相反, top-N性能可以通过基于准确性度量(如精度/召回率)的替代方法直接测量。

对几种最先进的推荐算法的广泛评估表明, 针对最小化RMSE而优化的算法在top-N推荐任务方面不一定像预期的那样执行。结果表明, RMSE的提高通常不会转化为精度的提高。特别是, 一个朴素的非个性化算法可以超越一些常见的推荐方法, 并且几乎可以与复杂算法的精度相媲美。另一个发现是, 极少数最受欢迎的商品会影响top-N名的表现。分析指出, 在评估top-N推荐任务上的推荐算法时, 应该仔细选择测试集, 以免准确度指标偏向非个性化解决方案。最后, 我们为从业者提供了两种协作过滤算法的新变体, 无论其RMSE如何, 在追求top-N推荐任务时, 它们都明显优于其他推荐算法, 并提供了额外的实用优势。考虑到这两种方法的简单性, 这让人感到惊讶。

类别和主题描述符

H.3.4[信息存储和检索]:系统和软件-用户配置文件和警报服务;性能评估(效率和效果);H.3.3[信息存储和检索]:信息搜索与检索-信息过滤

一般条款

算法、实验、测量、性能

1. 介绍

推荐系统的一个常见做法是通过诸如RMSE(均方根误差)之类的误差度量来评估它们的性能, RMSE捕获实际评级与系统预测评级之间的平均误差。然而, 在许多商业系统中, 只显示“最佳赌注”的推荐, 而预测的评分值不是[8]。也就是说, 系统会向用户推荐一些可能对他非常有吸引力的特定项目。虽然大多数文献关注的是方便的误差指标(RMSE、MAE), 但这些经典的误差标准并不能真正衡量top-N性能。最多, 它们可以作为真正的top-N体验的代理。top-N性能的直接评估必须通过基于准确性度量(例如, 召回率和精度)的替代方法来完成。

在本文中, 我们通过准确性指标评估了几种协同过滤算法在追求top-N推荐任务中的性能。评估与RMSE度量上相同方法的性能进行对比。已经在Netflix和Movielens数据集上执行了测试。

这项工作的贡献有三个方面:(i)我们表明误差指标和准确性指标之间没有微不足道的关系;(ii)我们提出了一种不偏向精度指标的测试集的仔细构建;(iii)我们引入现有算法的新变体, 提高top-N性能以及其他实际效益。

我们首先将一些最先进的算法(例如, 非对称SVD)与基于物品流行度的非个性化算法进行比较。令人惊讶的结果是, 非个性化算法在top-N推荐上的性能与复杂的个性化算法的性能相当, 而不管它们的RMSE如何。

然而, 非个性化的、基于流行度的算法只能提供琐碎的推荐, 既不会让用户感兴趣, 用户可能会因为推荐系统而感到厌烦和失望, 也不会让内容提供商感兴趣, 后者投资推荐系统是为了推高不太为人所知的物品的销量。出于这样的原因, 我们额外运行了一组实验, 以便在排除极受欢迎的物品的同时评估算法的性能。正如预期的那样, 所有算法的准确率都会下降, 因为推荐非平凡项目变得更加困难。然而, 不同算法的排名更符合我们的预期, 与

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

RecSys2010, 2010年9月26-30日, 西班牙巴塞罗那。版权
2010 ACM 978-1-60558-906-0/10/09...\$10.00。

非个性化方法的排名较低。因此，在评估top-N推荐任务中的算法时，我们建议仔细选择测试集，否则准确性指标会有严重偏差。

最后，在执行top-N推荐任务时，不需要精确的评级预测。我们提出了两种协作过滤算法的新变体，它们不是为了最小化RMSE而设计的，但在top-N推荐中始终优于其他推荐算法。当考虑到性能优于方法的简单和不那么传统的性质时，这一发现变得更加重要。

2. 测试方法

本研究采用的测试方法与[6]，特别是[10]中描述的方法相似。对于每个数据集，已知的评级被分成两个子集：训练集M和测试集T，测试集T只包含5星评级。因此，我们可以合理地声明，T包含与各自用户相关的项目。

从Netflix数据集创建M和T的详细过程与Netflix奖的数据集相似，保持了与其他研究论文发表的结果的兼容性[3]。Netflix发布了一个包含约1亿个收视率的训练数据集，简称训练数据集。除了训练集，Netflix还提供了一个验证集，称为探针集，包含140万个收视率。在本工作中，训练集M是原始的Netflix训练集，而测试集T包含了来自探针集(TI=384,573)的所有5星评分。不出所料，没有使用探针集进行训练。

我们对Movielens数据集采用了类似的程序[13]。为了创建探测集，我们从数据集中随机抽样1.4%的评分。训练集M包含了剩余的评级。测试集T包含来自探测集的所有5星评级。

为了测量召回率和准确率，我们首先在m中的评分上训练模型，然后，对于用户u在T中的每个项目，我给它打了5星评分：

- (i)我们随机选择了1000个未被用户u评分的额外物品。我们可以假设它们中的大多数对用户u来说都不感兴趣。
- (ii)预测测试项目i和额外的1000个项目的评分。
- (iii)我们根据所有1001个项目的预测评级对它们进行排序，形成一个排序列表。让p表示测试项目i在这个列表中的排名。最好的结果对应于测试项目i排在所有随机项目之前的情况(即p = 1)。
- (iv)我们通过从列表中选择N个排名靠前的项目来形成一个top-N推荐列表。如果p ≤ N，我们有一个hit(即，测试项目i被推荐给用户)。否则我们会有失误。命中的机会随着N的增加而增加。当N = 1001时，我们总是有命中。

查全率和查准率的计算如下。对于任何单个测试用例，我们有一个相关的项目(被测试的项目i)。根据定义，单个测试的召回率可以假设值为0(在未命中的情况下)或1(在命中的情况下)。类似地，精确度可以假设这两个值中的任何一个

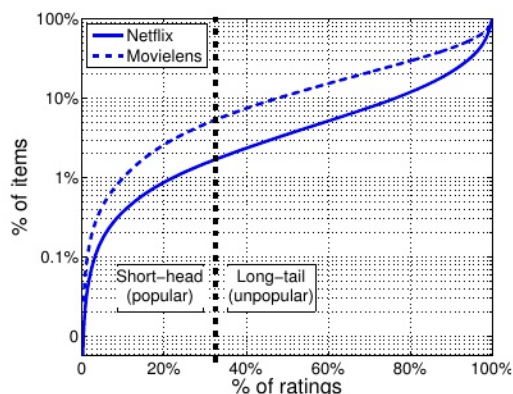


图1:Netflix(实线)和Movielens(虚线)的收视率分布。物品按受欢迎程度排序(最受欢迎的在底部)。

0或1/N。通过对所有测试用例取平均值来定义整体的召回率和准确率：

$$\text{recall}(N) = \frac{\# \text{hits}}{|T|}$$
$$\text{precision}(N) = \frac{\# \text{hits}}{N \cdot |T|} = \frac{\text{recall}(N)}{N}$$

其中|T|是测试评级的数量。注意，假设所有的1000个随机项目都与用户u无关，这往往会低估计算的召回率和准确率相对于真实的召回率和准确率。

2.1 热门物品vs.长尾

根据众所周知的适用于许多商业系统的评分物品的长尾分布，大部分评分都浓缩在一小部分最受欢迎的物品[1]中。

图1绘制了Netflix和Movielens数据集的经验评分分布。纵轴上的项目按照受欢迎程度排序，最受欢迎的在底部。我们观察到，Netflix收集的33%的评分只涉及1.7%的最受欢迎的项目(即302个项目)。我们将这个非常受欢迎的小集合称为短头(short-head)，剩余的不太受欢迎的集合(约占总数的98%)称为长尾[5]。我们还注意到，Movielens的评分分布比Netflix的要稍微短一些：短头(33%的评分)涉及5.5%的热门项目(即213个项目)。

推荐流行物品是微不足道的，不会给用户和内容提供者带来太多好处。另一方面，推荐不太知名的物品会给用户带来新鲜感和意外收获，但这通常是一项更困难的任务。在这项研究中，我们的目标是评估推荐算法在推荐非平凡项目方面的准确性。为此目的，测试集T被进一步划分为两个子集 T_{head} 和 T_{long} ，这样， T_{head} 中的项目处于短头，而 T_{long} 中的项目处于分布的长尾。

3. 协作算法

大多数推荐系统都是基于协同过滤(CF)，其中推荐仅依赖于过去的用户

行为(在这里称为“评级”，尽管这种行为可以包括购买、租赁和点击等项目的其他用户活动)，而不考虑领域知识。有两种主要的CF方法:(i)邻域方法和(ii)潜在因素方法。

邻域模型代表了CF最常用的方法，它们基于用户或项目之间的相似性。例如，两个用户是相似的，因为他们对相同的一组物品进行了相似的评分。可以在物品之间定义相似度的双重概念。

隐因子方法通过减少隐因子的数量，将用户和项目建模为相同“隐因子”空间中的向量。在这样的空间中，用户和物品是直接可比较的:用户 u 对物品 i 的评分是通过相关潜在因子向量之间的接近程度(例如内积)来预测的。

3.1 非个性化模型

非个性化推荐向任何用户呈现一个预定义的、固定的项目列表，而不管他/她的偏好如何。这样的算法可以作为更复杂的个性化算法的基线。

一个简单的估计规则，称为电影平均(MovieAvg)，推荐具有最高平均评分的前 n 项。用户 u 对物品 i 的评分被预测为社区对物品 i 表示的平均评分，而不考虑 u 给出的评分。

一个类似的预测模式，用Top Popular (Top-pop)表示，推荐受欢迎程度最高(评级数量最多)的前 n 项。注意，在这种情况下，无法推断出用户 u 对物品 i 的评分，但是这个算法的输出只是一个物品的排序列表。因此，RMSE或其他误差度量是不适用的。

3.2 社区模型

邻域模型的预测基于用户或项目之间的相似性关系。

以用户-用户相似性为中心的算法根据与用户相似的用户对该项目的评分来预测用户的评分。另一方面，以项目-项目相似度为中心的算法根据用户自己对相似项目的评分来计算用户对项目的偏好。后者通常是首选的方法(例如，[15])，因为它通常在RMSE方面表现更好，同时更具可扩展性。这两种优势都与物品数量通常小于用户数量的事实有关。item-item算法的另一个优点是，向特定用户推荐的背后的推理可以根据他/她之前评级的项目来解释。此外，基于项目(而不是用户)来建立系统参数，可以无缝处理系统新加入的用户和评分。出于这样的原因，我们专注于物品-物品邻域算法。

项目 i 和项目 j 之间的相似度是通过用户对项目 i 和 j 相似评分的倾向来衡量的。它通常基于余弦、调整余弦或(最常见的)皮尔逊相关系数[15]。

物品之间的相似度是根据共同评分者计算的。在非常稀疏的数据集的典型情况下，很可能有些项目对的支持度很差，导致不可靠的相似性度量。出于这样的原因，如果 n_{ij} 表示共同评分者的数量， s_{ij} 表示项目 i 和项目 j 之间的相似度，我们可以定义缩减后的相似度 d_{ij}

作为系数 $d_{ij}=s_{ij}/n_{ij}$ 其中 λ_i 是一个缩小的 $n_{ij} + \lambda_i$ 因素[10]。 λ_i 的典型值为100。

邻域模型通过kNN (k-近邻)方法进一步增强。在预测评分 r_{ui} 时，我们只考虑与 i 最相似的 k 个由 u 评分的项目。我们用 $D^k(u;i)$ 表示最相似项目的集合。kNN方法丢弃了与目标项目相关性较差的项目，从而降低了噪声，提高了推荐的质量。

在比较和求和不同的评分之前，建议去除掩盖了项目之间更基本关系的不同偏差。这样的偏差包括项目效应，它代表了某些项目倾向于获得比其他项目更高的评分的事实。它们还包括用户效应(usereffects)，代表某些用户的评分倾向高于其他用户。更精细的偏差计算，也会估计时间效应[11]，但这超出了这项工作的范围。我们将遵循[10]的静态项目和用户偏差作为基线。形式上，与用户 u 对项目 i 的评分相关的偏差用 b_{ui} 表示。

item-item kNN方法预测残差评级 $r_{ui}-b_{ui}$ 为相似项目残差评级的加权平均值:

$$\hat{r}_{ui} = b_{ui} + \frac{\sum_{j \in D^k(u;i)} d_{ij} (r_{uj} - b_{uj})}{\sum_{j \in D^k(u;i)} d_{ij}} \quad (1)$$

以下我们将该模型称为相关邻域(CorNgbr)，其中 s_{ij} 为皮尔逊相关系数。

3.2.1 非归一化余弦邻域

注意，在(1)中，分母迫使预测评级值的 p 落在正确的范围内，例如，[1...5]对于一个典型的星级评级系统。然而，对于top-N推荐任务，不需要精确的评分值。我们只是想根据物品对用户的吸引力来排序。在这种情况下，我们可以通过去除分母来简化公式。这样做的一个好处是具有许多相似邻居的项目的排名更高(即高 $j \in D^k(u;i)$ 的 d_{ij})，在这种情况下我们对推荐有更高的置信度。因此，我们建议通过以下系数(r_{ui})对物品进行排序

$$\hat{r}_{ui} = b_{ui} + \sum_{j \in D^k(u;i)} d_{ij} (r_{uj} - b_{uj}) \quad (2)$$

在这里 r_{ui} 并不代表一个适当的评级，而是一个度量用户 u 和物品 i 之间关联的指标。我们应该注意，类似的非归一化邻域规则也被其他人提到[7,10]。

在我们的实验中，通过计算 s_{ij} 作为余弦相似度，获得了准确度指标方面的最佳结果。与皮尔逊相关(Pearson correlation)仅在共同评分者共享的评分上计算不同，项目 i 和 j 之间的余弦系数是在所有评分上计算的(将缺失值作为零)，即: $\cos(i, j) = \vec{i} \cdot \vec{j} / (\|\vec{i}\|_2 \cdot \|\vec{j}\|_2)$ 。我们用非归一化余弦邻域(NNCosNgbr)来表示这种模型。

3.3 隐语义模型

最近，几种基于隐语义模型的推荐算法被提出。其中大部分是

基于分解用户-物品评级矩阵[12], 也被非正式地称为SVD模型, 经过相关的奇异值分解。

SVD模型的关键思想是将用户-物品评级矩阵分解为两个较低秩矩阵的乘积, 一个包含所谓的“用户因素”, 而另一个包含所谓的“物品因素”。因此, 每个用户 u 都用一个 f 维的用户因子向量 $\mathbf{p}_u \in \mathbb{R}^f$ 表示。同样, 每个物品 i 都用一个物品因素向量 $\mathbf{q}_i \in \mathbb{R}^f$ 表示。用户 u 对物品 i 给出的评分预测被计算为相关因子向量之间的内积(调整偏差), 即:

$$\hat{r}_{ui} = b_{ui} + \mathbf{p}_u \mathbf{q}_i^T \quad (3)$$

由于传统的SVD在存在未知值时是未定义的。缺失评级已经提出了几种解决方案。早期的工作通过用基线估计(例如[16])填补缺失评级来解决这个问题。然而, 这导致了一个非常大、密集的用户评分矩阵, 其因子分解在计算上变得不可行。最近的工作通过适当的目标函数直接在已知评分上学习因子向量, 从而最小化预测误差。所提出的目标函数通常是正则化的, 以避免过拟合(例如[14])。通常, 梯度下降被应用于最小化目标函数。

与邻域方法一样, 本文侧重于将用户表示为项目特征组合的方法, 而不需要任何特定于用户的参数化。这些方法的优点是, 它们可以为系统的新用户创建推荐, 而无需重新评估参数。很可能, 他们可以立即将推荐调整为刚刚输入的评分, 为用户的行为提供即时反馈。最后, 这样的方法可以根据用户之前评分的项目来解释他们的推荐。

因此, 我们尝试了一个强大的矩阵分解模型, 它确实将用户表示为项目特征的组合。该方法被称为不对称svd (AsySVD), 据报道在Netflix数据集上达到0.9000的RMSE [10]。

此外, 我们还试验了一种增强的矩阵分解方法, 称为SVD++[10], 它代表了rmse优化分解方法中的最高质量, 尽管用户不再被表示为项目特征的组合;参见[10]。

3.3.1 PureSVD

在追求top-N推荐任务时, 我们只对正确的项目排名感兴趣, 而不关心准确的评级预测。这给了我们一些灵活性, 比如将用户评分矩阵中的所有缺失值都考虑为0, 尽管超出了1-5星的评分范围。在预测能力方面, 零的选择并不是很重要, 我们已经收到了较高估算值的类似结果。重要的是, 现在我们可以利用现有的高度优化的软件包在稀疏矩阵上执行传统的SVD, 这变得可行, 因为所有矩阵条目现在都不丢失了。因此, 用户评分矩阵 $\mathbf{R}$ 是通过因子分解[2]估计的:

$$\hat{\mathbf{R}} = \mathbf{U} \cdot \mathbf{\Sigma} \cdot \mathbf{Q}^T \quad (4)$$

其中, $\mathbf{U}$ 是 $n \times f$ 标准正交矩阵, $\mathbf{Q}$ 是 $m \times f$

Dataset	Users	Items	Ratings	Density
MovieLens	6,040	3,883	1M	4.26%
Netflix	480,189	17,770	100M	1.18%

表1:MovieLens和Netflix的统计属性。

标准正交矩阵, 以及 $\mathbf{\Sigma}$ 是一个包含前 f 个奇异值的 $f \times f$ 对角矩阵。

为了证明归零的简单性, 我们应该提到我们使用了一个非多线程SVD包(SVDLIBC, 基于SVDPACKC库[4]), 它在一台i7 PC ($f = 150$)上10分钟内分解了17770部Netflix电影数据集的480K用户。

让我们定义 $\mathbf{P} = \mathbf{U} \cdot \mathbf{\Sigma}$, 这样 $\mathbf{P}$ 的第 u 行表示用户因子向量 $\mathbf{p}_u$, 而 $\mathbf{Q}$ 的第 i 行表示物品因子向量 $\mathbf{q}_i$ 。相应地, r_{ui} 的计算方法类似于(3)。

此外, 由于 $\mathbf{U}$ 和 $\mathbf{Q}$ 具有标准正交列, 我们可以直接推导出:

$$\mathbf{P} = \mathbf{U} \cdot \mathbf{\Sigma} = \mathbf{R} \cdot \mathbf{Q} \quad (5)$$

其中 $\mathbf{R}$ 为用户评分矩阵。因此, 通过用 $\mathbf{r}_u$ 表示用户评分矩阵的第 u 行——即用户 u 的评分向量, 我们可以重写预测规则

$$\hat{r}_{ui} = \mathbf{r}_u \cdot \mathbf{Q} \cdot \mathbf{q}_i^T \quad (6)$$

注意, 与(1)类似, 在稍微滥用符号的情况下, 符号 r_{ui} 并不完全是一个有效的评分值, 而是用户 u 和物品 i 之间的关联度量。

在下文中, 我们将把这个模型称为PureSVD。与item-item kNN和AsySVD一样, PureSVD提供了将用户表示为项目特征的组合(通过Eq.(5))的所有好处, 而不需要任何特定于用户的参数化。它还提供了方便的优化, 不需要调优学习常数。

4. 结果

在本节中, 我们将介绍第3节中提出的推荐算法在两个标准数据集上的质量:MovieLens[13]和Netflix[3]。两者都是公开可用的电影评分数据集。收集到的评分等级为1-5星。表1总结了它们的统计属性。

我们使用了第2节中定义的方法来评估6种推荐算法。前两个——MovieAvg和TopPop——都是非个性化算法, 我们希望它们的表现能被任何推荐算法超越。第三种预测规则——cornbr——是一种基于邻域的优化算法, 可能是协同过滤文献中最流行的。第四种算法是cornbr的一种变体- NNCosNgb r -它是两个提出的面向精度度量的算法之一。第五是包含200个因子的AsySVD潜在因子模型。第六种是200-D SVD++模型, 在RMSE方面是最强大的潜在因素模型之一。最后, 我们考虑潜在因素模型的变体, PureSVD, 它以两种配置显示:一种具有较少的潜在因素(50), 另一种具有较多的潜在因素(MovieLens为150,Netflix数据集为300)。

其中三种算法——toppop、NNCosNgb r和PureSVD——从误差最小化的角度来看是不明智的, 不能通过RMSE度量来评估。其他四种

优化算法以获得最佳的RMSE结果，其在Netflix测试集上的RMSE得分如下:MovieAvg为1.053,cornbr为0.9406,AsySVD为0.9000,SVD++为0.8911[10]。

对于每个数据集，我们在完整测试集上进行了一组实验，在长尾测试集上进行了一组实验。我们将召回率报告为N的函数(即推荐项目的数量)，并将精度报告为召回率的函数。关于召回率(N)，我们在[1..]范围内放大了N。20)。对于典型的top-N推荐任务，可以忽略较大的N值。的确，一部吸引人的电影被放置在前100名和前200名之间并没有区别，因为在这两种情况下，它都不会呈现给用户。

4.1 Movielens数据集

图2报告了在整个测试集上Movielens数据集上算法的性能。很明显，这些算法在top-N精度方面存在显著的性能差异。例如，在N = 10时，AsySVD的召回率约为0.28，即模型有28%的概率将一部吸引人的电影放在前10名中。令人惊讶的是，非个性化TopPop的召回率与AsySVD非常相似(例如，在N = 10时召回率约为0.29)。准确度方面最好的算法是不面向rmse的NNCosNgbr和PureSVD，在N = 10时，召回率分别达到约0.44和0.52。至于后者，这意味着大约50%的5星级电影呈现在top-10推荐中。在面向rmse的算法族中，最好的算法是SVD++，召回率接近NNCosNgbr。

图2(b)证实，PureSVD在精度指标方面也优于其他算法，其次是另一种非面向rmse的算法nnccosngbr。每一行代表算法在给定召回率下的精度。例如，当召回率约为0.2时，NNCosNgbr的精度约为0.12。同样，TopPop的性能与最先进的算法(如AsySVD)保持一致。我们应该注意到广泛使用的cornbr算法的总体表现不佳，其性能与非常幼稚的MovieAvg一致。我们还应该注意到，对于小召回值，SVD++的精度与PureSVD50具有竞争力。

TopPop的奇怪和意外的结果激发了第二组实验，在长尾项目上完成，其结果如图3所示。作为提醒，现在我们从考虑中排除了非常受欢迎的物品。在这里，几种推荐算法的排序更符合我们的预期。事实上，非个性化的TopPop的召回率和准确率急剧下降，在前20个位置内推荐一部五星级电影的可能性很小。然而，即使关注长尾，最好的算法仍然是PureSVD，其在N = 10时的召回率约为40%。请注意，在完整测试集的情况下，PureSVD的最佳性能是在50个潜在因素的情况下，而在这里，潜在因素的数量更大，即150个，才能达到最佳性能。NNCosNgbr的性能现在明显低于PureSVD，而SVD++在面向rmse算法中是最好的。

4.2 Netflix数据集

与Movielens的结果类似，图

4和图5显示了算法在Netflix数据集上的性能。和之前一样，我们同时关注完整测试集和长尾测试集。

再一次，非个性化的TopPop在包含2%的头部项目时显示出令人惊讶的好结果，超过了广受欢迎的cornbr。然而，更大的AsySVD和sdv++可能对Netflix的数据进行了更好的调整，它们的表现略优于Top-Pop。还要注意，AsySVD现在与SVD++一致。

与Movielens的经验一致，在召回率和精度方面表现最好的算法仍然是非面向rmse的PureSVD。至于另一种非面向rmse的NNCosNgbr，情况就复杂了。当包括头部项目时，它仍然优于面向rmse的算法，但当排除前2%最受欢迎的项目时，它的性能略低于它们。

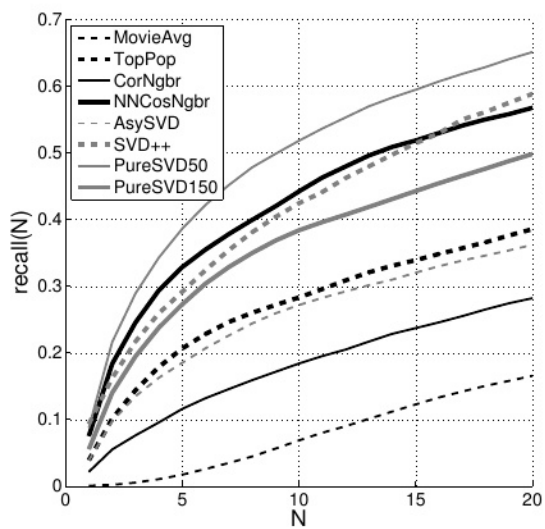
Netflix数据集上常用的cornbr的行为非常令人惊讶。虽然它在整个测试集上的表现明显落后于其他算法，但当专注于较长的尾部时，它就成为了表现最好的算法之一。事实上，当所有算法从完整测试集传递到长项目测试集时，它们的精度和召回率都会降低，而cornbr在推荐长尾项目时显得更准确。毕竟，考虑到长尾项目的重要性，cornbr方法被广泛接受可能是有原因的。

5. 讨论-PureSVD

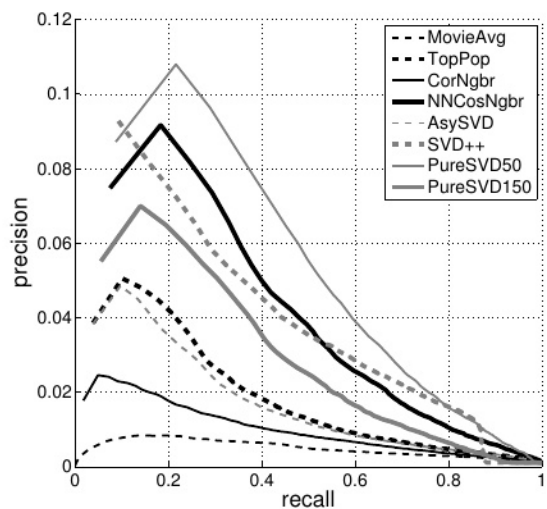
在Movielens和Netflix的数据集上，无论是否包含头部物品，PureSVD始终是表现最好的，击败了更详细和复杂的潜在因素模型。考虑到它的简单性，以及RMSE优化方面的糟糕设计，我们没有想到会有这样的结果。事实上，我们认为这对推荐系统的从业者来说是一个好消息，因为PureSVD结合了多种优势。首先，它非常容易编写代码，不需要调整学习常数，并且完全依赖于现成的优化SVD包。这在离线和在线模式下都具有良好的计算性能。PureSVD还方便地将用户表示为项目特征的组合(公式(6))，为设计师提供了处理新用户、现有用户新评分和解释生成推荐背后原因的灵活性。

在Movielens和Netflix数据集上观察到一个有趣的发现，当移动到长尾项目时，准确性随着PureSVD模型维度的提高而提高。这可能与以下事实有关:PureSVD的第一个潜在因素捕获了最受欢迎的物品的属性，而其他潜在因素则代表了与不受欢迎的物品相关的更精细的特征。因此，当从业者使用PureSVD时，他们应该选择它的维度，同时考虑到潜在因素的数量对长尾项目质量的影响不同于正面项目。

我们想提供一个解释，为什么PureSVD可以始终提供更好的top-N结果比最好的rmse提炼潜在因素模型。这可能与RMSE测试的限制有关，它只关注用户提供给系统的评级。这样，RMSE(或MAE)是在一个只包含用户选择评价的项目的测试集上测量的，而完全忽略了对该方法的任何评估

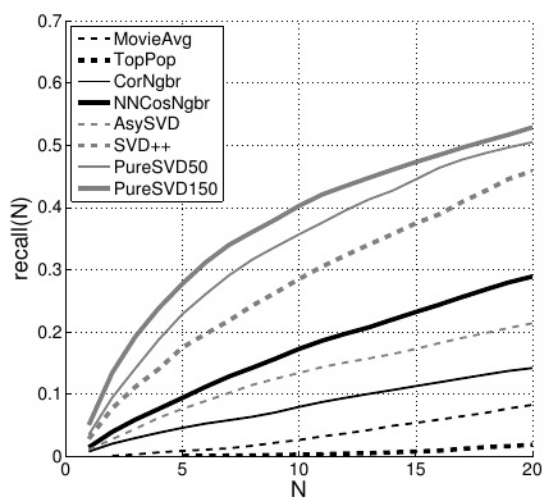


(a) recall

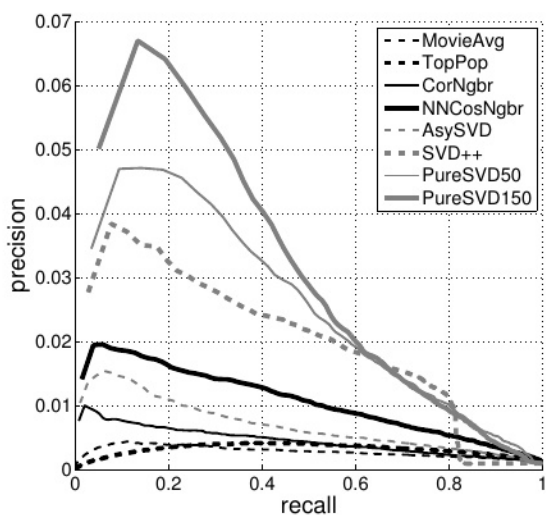


(b) precision vs recall

图2:电影:(a)所有项目的召回率- n和(b)精度与召回率。

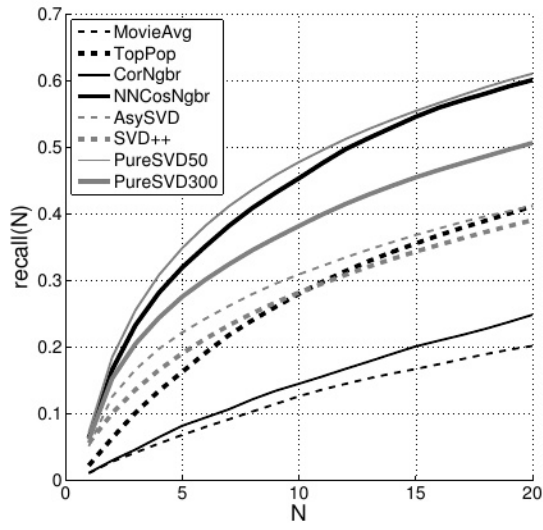


(a) recall

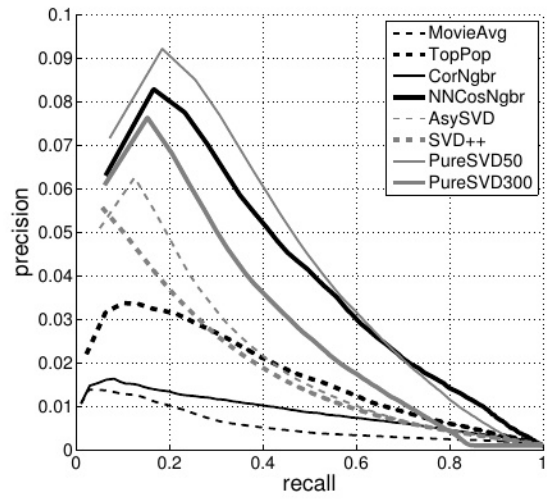


(b) precision vs recall

图3:电影:(a)召回率- n和(b)长尾上的精确率-召回率(94%的项目)。

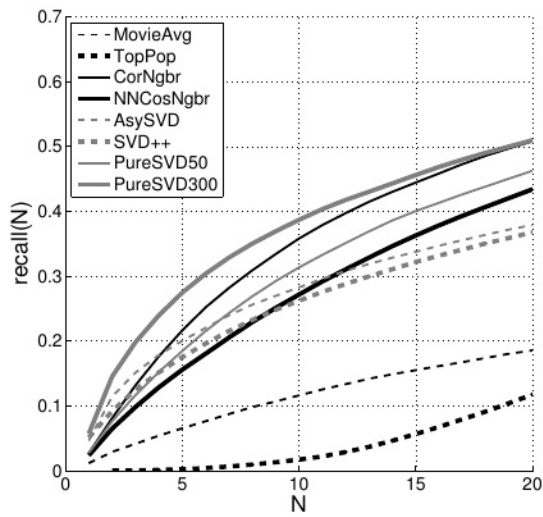


(a) recall

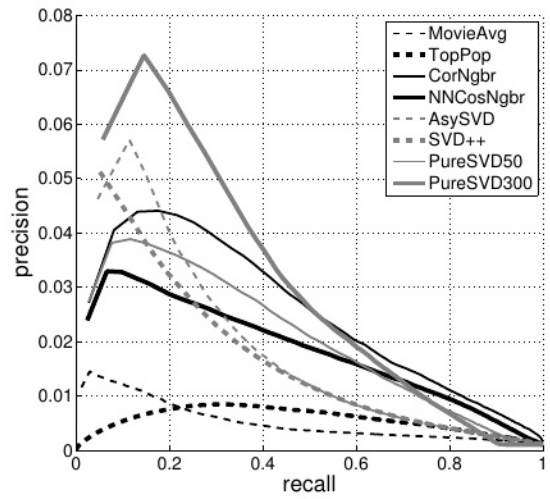


(b) precision vs recall

图4:Netflix:(a)所有项目的召回率和(b)精确率与召回率。



(a) recall



(b) precision vs recall

图5:Netflix:(a)召回率- n和(b)长尾(98%的项目)的精度与召回率。

用户从未评分过的物品。这种测试模式对于面向rmse的模型来说是个好兆头, rmse只在已知的评级上进行训练, 而在很大程度上忽略了缺失的条目。然而, 这样的测试方法忽略了很多现实, 即所有项目都应该被计算, 而不仅仅是那些用户过去实际评分的项目。所提出的基于Top-N的准确性度量确实在这方面做得更好, 通过在测试阶段直接涉及所有可能的项目(包括未评级的项目)。这可能解释了PureSVD在训练阶段的优异表现, 它考虑了所有可能的用户-项目对(不管评级可用性)。

我们对从业者的一般建议是将PureSVD视为推荐算法。仍然有一些未被探索的方法可以改善PureSVD。首先, 可以优化缺失条目的估算值。通过翻译所有给定的分数, 仍然可以直接使用现成的稀疏SVD解算器(通常假设默认值为零)。例如, 估算值为3而不是0将通过将给定的星级评级从[1...5]范围转化为[-2...2]范围内。其次, 可以对输入值给予较低的置信度, 这样SVD将更多地强调实际评级。关于如何实现这一点的解释, 请考虑[9]。然而, 我们期望这样的置信度加权会显著增加离线训练的时间复杂度。

6. 结论

长期以来, 推荐人的评估分为准确性指标(例如, 精度/召回率)和错误指标(特别是RMSE和MAE)。数学上的便利性和与形式化优化方法的适应度, 使得像RMSE这样的误差度量更受欢迎, 并且它们确实在文献中占据主导地位。然而, 众所周知, 准确性度量可能是一个更自然的尺度, 因为它们直接评估top-N推荐的质量。

这项工作表明, 通过广泛的实证研究, 方便的假设, 如RMSE误差度量可以作为top-N精度的良好代理, 充其量是值得怀疑的。误差指标和精度指标之间不存在单调关系。这可能需要对top-N系统的优化目标进行重新评估。好的一面是, 我们提出了已知算法的简单而有效的变体, 这些变体在RMSE方面是无用的, 但在追求top-N精度时却提供了卓越的结果。

顺便说一下, 我们还讨论了在设计用于进行top-N精度评估的测试集时可能存在的缺陷。特别是, 一个粗心的测试集构造将使召回率和精度强烈偏向于非个性化算法。我们采用的一个简单的解决方案是, 将非常受欢迎的项目从测试集中排除在外(同时保留了98%的项目)。由此产生的测试集, 强调了相当重要的非微不足道的项目, 似乎与我们的期望形成了更好的形状。首先, 它正确地显示了非个性化算法的较低值。其次, 它显示了广泛使用的基于相关性的kNN方法的良好行为, 否则(当在完整的项目集上进行评估时)表现出极其糟糕的结果, 强烈地与公认的实践相对抗。

7. 引用

- [1] C. Anderson. The Long Tail: Why the Future of Business Is Selling Less of More. Hyperion, July 2006.

- [2] R. Bambini, P. Cremonesi, and R. Turrin. Recommender Systems Handbook, chapter A Recommender System for an IPTV Service Provider: a Real Large-Scale Production Environment. Springer, 2010.
- [3] J. Bennett and S. Lanning. The Netflix Prize. Proceedings of KDD Cup and Workshop, pages 3–6, 2007.
- [4] M. W. Berry. Large-scale sparse singular value computations. The International Journal of Supercomputer Applications, 6(1): 13–49, Spring 1992.
- [5] O. Celma and P. Cano. From hits to niches? or how popular artists can bias music recommendation and discovery. Las Vegas, USA, August 2008.
- [6] P. Cremonesi, E. Lentini, M. Matteucci, and R. Turrin. An evaluation methodology for recommender systems. 4th Int. Conf. on Automated Solutions for Cross Media Content and Multi-channel Distribution, pages 224–231, Nov 2008.
- [7] M. Deshpande and G. Karypis. Item-based top-n recommendation algorithms. ACM Transactions on Information Systems (TOIS), 22(1):143–177, 2004.
- [8] J. Herlocker, J. Konstan, L. Terveen, and J. Riedl. Evaluating collaborative filtering recommender systems. ACM Transactions on Information Systems (TOIS), 22(1):5–53, 2004.
- [9] Y. Hu, Y. Koren, and C. Volinsky. Collaborative filtering for implicit feedback datasets. Data Mining, IEEE International Conference on, 0:263–272, 2008.
- [10] Y. Koren. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In KDD '08: Proceeding of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining, pages 426–434, New York, NY, USA, 2008. ACM.
- [11] Y. Koren. Collaborative filtering with temporal dynamics. In KDD '09: Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining, pages 447–456, New York, NY, USA, 2009. ACM.
- [12] Y. Koren, R. M. Bell, and C. Volinsky. Matrix factorization techniques for recommender systems. IEEE Computer, 42(8): 30–37, 2009.
- [13] B. Miller, I. Albert, S. Lam, J. Konstan, and J. Riedl. MovieLens unplugged: experiences with an occasionally connected recommender system. Proceedings of the 8th international conference on Intelligent user interfaces, pages 263–266, 2003.
- [14] A. Paterek. Improving regularized singular value decomposition for collaborative filtering. Proceedings of KDD Cup and Workshop, 2007.
- [15] B. Sarwar, G. Karypis, J. Konstan, and J. Reidl. Item-based collaborative filtering recommendation algorithms. 10th Int. Conf. on World Wide Web, pages 285–295, 2001.
- [16] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl. Application of Dimensionality Reduction in Recommender System-A Case Study. Defense Technical Information Center, 2000.